

.\11.py

```
import numpy as np

def sigmoid(x):
    return 1 / (1 + np.exp(-x))

class Perceptron:
    def __init__(self, n):
        self.w = np.random.rand(n)
        self.b = np.random.rand()

    def forward(self, x):
        return sigmoid(np.dot(x, self.w) + self.b)

    def train(self, X, y, epochs=1000, lr=0.1):
        for _ in range(epochs):
            for i in range(len(X)):
                error = y[i] - self.forward(X[i])
                self.w += lr * error * X[i]
                self.b += lr * error

X = np.array([[0,0],[0,1],[1,0],[1,1]])

data = {
    "AND": np.array([0,0,0,1]),
    "OR": np.array([0,1,1,1])
}

for gate, y in data.items():
    p = Perceptron(2)
    p.train(X, y)

    print(f"\n{gate} Gate:")
    for x in X:
        print(x, "->", round(p.forward(x)))

# AND Gate:
# [0 0] -> 0
# [0 1] -> 0
# [1 0] -> 0
# [1 1] -> 1

# OR Gate:
# [0 0] -> 0
# [0 1] -> 1
# [1 0] -> 1
# [1 1] -> 1
```

.\1hillClimbing.py

```
import random

def objective_function(x):
    return -x ** 2 + 5

def hill_climbing(start_x, step_size, max_iterations):
    current_x = start_x
    current_score = objective_function(current_x)

    for i in range(max_iterations):
        new_x = current_x + random.uniform(-step_size, step_size)
        new_score = objective_function(new_x)

        print(f"Iteration {i + 1}: x = {current_x:.4f}, f(x) = {current_score:.4f}")

        if new_score > current_score:
            current_x = new_x
            current_score = new_score
        else:
            pass

    print("\nFinal Solution:")
    print(f"x = {current_x:.4f}, f(x) = {current_score:.4f}")
    return current_x, current_score

best_x, best_score = hill_climbing(start_x=0.1, step_size=0.05, max_iterations=5)
```

```
# # User Input
# start = int(input("Enter starting point: "))
# step_size = int(input("Enter step size: "))
# max_iterations = int(input("Enter maximum iterations: "))

# # Run algorithm
# best_x, best_value = hill_climbing(
#     start,
#     step_size,
#     max_iterations
# )

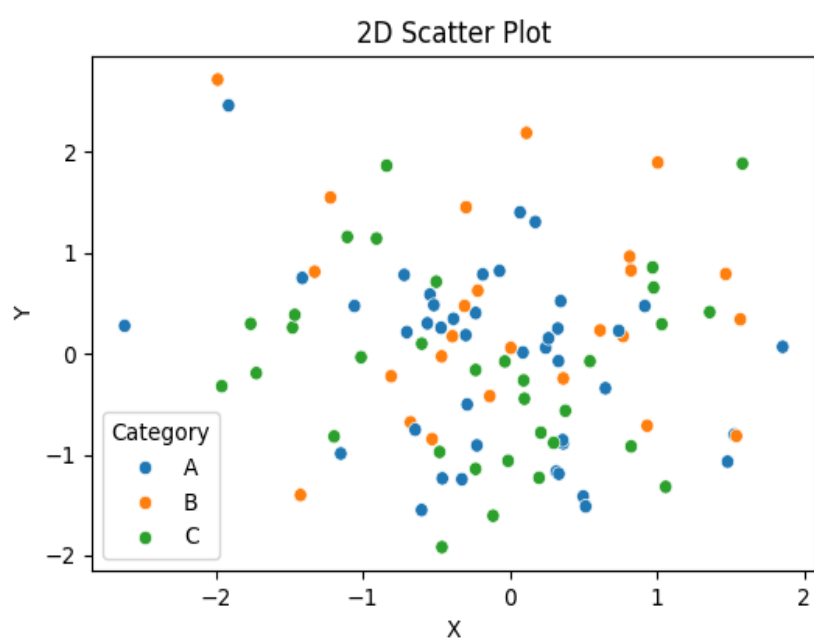
# # Display final answer
# print("\nBest Solution :", best_x)
# print("Maximum Value :", best_value)
```

The scatter plot visualizes the relationship between two dimensions (X and Y) of an n-dimensional

```
"""
      X      Y      Z Category
0   0.496714 -1.415371 0.357787      A
1  -0.138264 -0.420645 0.560785      B
2   0.647689 -0.342715 1.083051      A
3   1.523030 -0.802277 1.053802      A
4  -0.234153 -0.161286 -1.377669      C
..      ...      ...      ...
95 -1.463515 0.385317 -0.692910      C
96 0.296120 -0.883857 0.899600      C
97 0.261055 0.153725 0.307300      A
98 0.005113 0.058209 0.812862      B
99 -0.234587 -1.142970 0.629629      C
```

```
[100 rows x 4 columns]
Iteration 1: x = 0.1000, f(x) = 4.9900
Iteration 2: x = 0.1000, f(x) = 4.9900
Iteration 3: x = 0.1000, f(x) = 4.9900
Iteration 4: x = 0.0905, f(x) = 4.9918
Iteration 5: x = 0.0894, f(x) = 4.9920
```

```
Final Solution:
x = 0.0807, f(x) = 4.9935"""
```



.\1\scatterPlot.py

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

n = 100

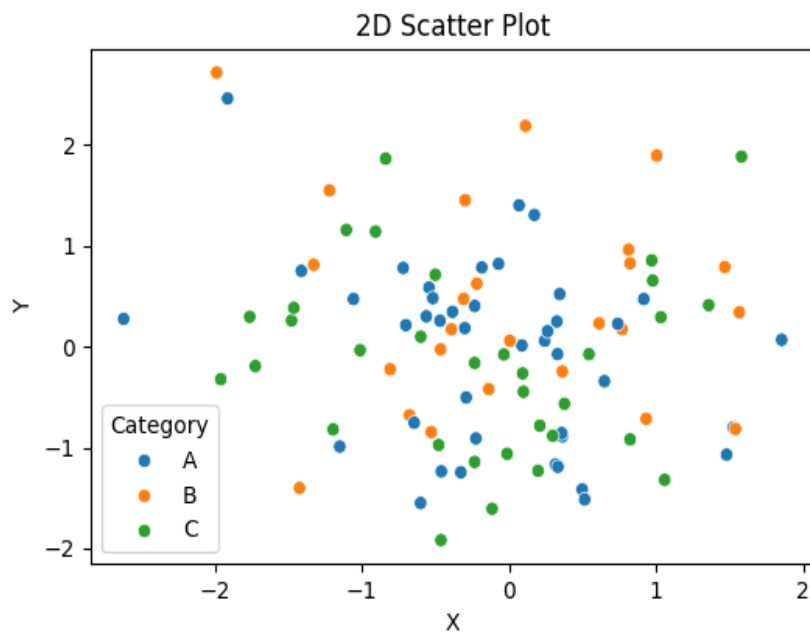
data = pd.DataFrame()

for c in ['X', 'Y']:
    data[c] = np.random.normal(size=n)

data['Category'] = np.random.choice(['A', 'B', 'C'], n)

sns.scatterplot(data=data, x='X', y='Y', hue='Category')

plt.title("Scatter Plot")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```



.\10\built_in.py

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA

iris = load_iris()
X = iris.data
y = iris.target
target_names = iris.target_names

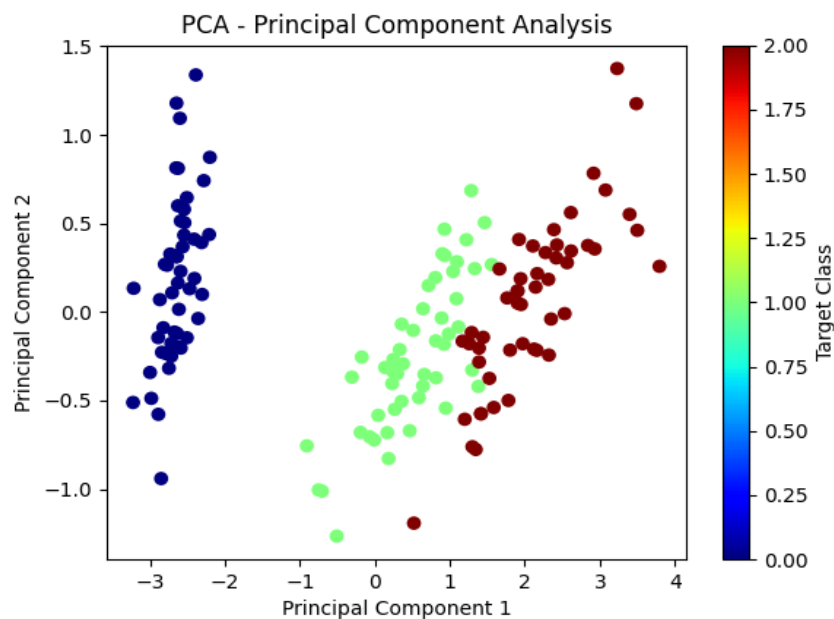
print("Original Data Shape:", X.shape)

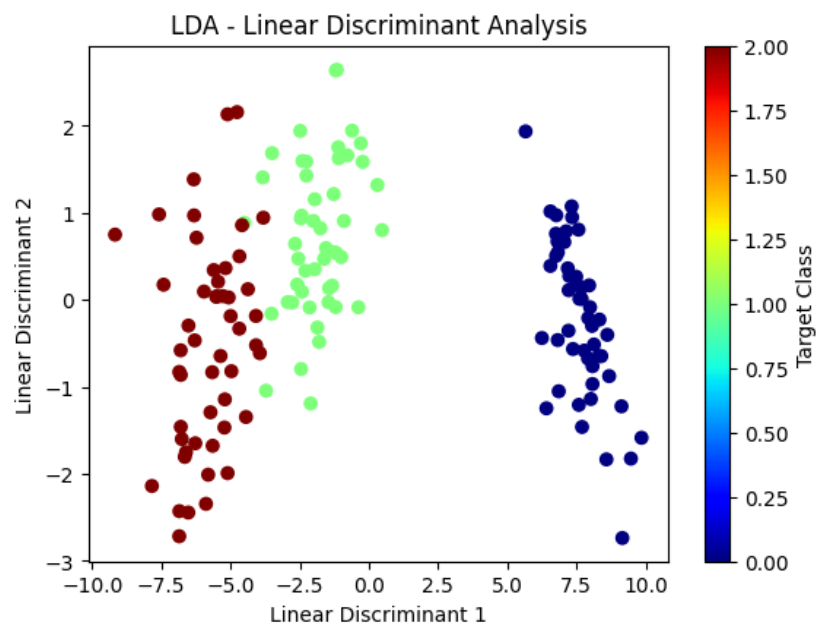
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
print("PCA Transformed Shape:", X_pca.shape)

plt.scatter(X_pca[:, 0], X_pca[:, 1], c=y, cmap='jet')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('PCA - Principal Component Analysis')
plt.colorbar(label='Target Class')
plt.show()

lda = LDA(n_components=2)
X_lda = lda.fit_transform(X, y)
print("LDA Transformed Shape:", X_lda.shape)

plt.scatter(X_lda[:, 0], X_lda[:, 1], c=y, cmap='jet')
plt.xlabel('Linear Discriminant 1')
plt.ylabel('Linear Discriminant 2')
plt.title('LDA - Linear Discriminant Analysis')
plt.colorbar(label='Target Class')
plt.show()
```





.\2\1a.py

```
from queue import PriorityQueue

def best_first_search(graph, heuristic, start, goal):
    pq = PriorityQueue()

    # (heuristic value, current node, path till current node)
    pq.put((heuristic[start], start, [start]))

    visited = set()

    while not pq.empty():
        h, current, path = pq.get()

        if current in visited:
            continue

        print("Visiting:", current)

        visited.add(current)

        if current == goal:
            return path

        for neighbor in graph[current]:
            if neighbor not in visited:
                pq.put((heuristic[neighbor], neighbor, path + [neighbor]))

    return None

graph = {}
heuristic = {}
nodes = []

n = int(input("Enter number of nodes: "))

for i in range(n):
    node = input("Enter node name: ")
    nodes.append(node)

    neighbors = input(f"Enter neighbors of {node} separated by comma: ")

    if neighbors.strip() == "":
        graph[node] = []
    else:
        graph[node] = [x.strip() for x in neighbors.split(",")]

print("\nEnter heuristic values:")

for node in nodes:
    heuristic[node] = int(input(f"Enter heuristic for {node}: "))

start = input("\nEnter start node: ")
goal = input("Enter goal node: ")

path = best_first_search(graph, heuristic, start, goal)

if path:
    print("\nPath found:", " -> ".join(path))
else:
    print("\nNo path found")

# Example Input:
#
# Enter number of nodes: 7
#
# Enter node name: A
# Enter neighbors of A separated by comma: B,C
#
# Enter node name: B
# Enter neighbors of B separated by comma: D,E
#
# Enter node name: C
# Enter neighbors of C separated by comma: F
#
```

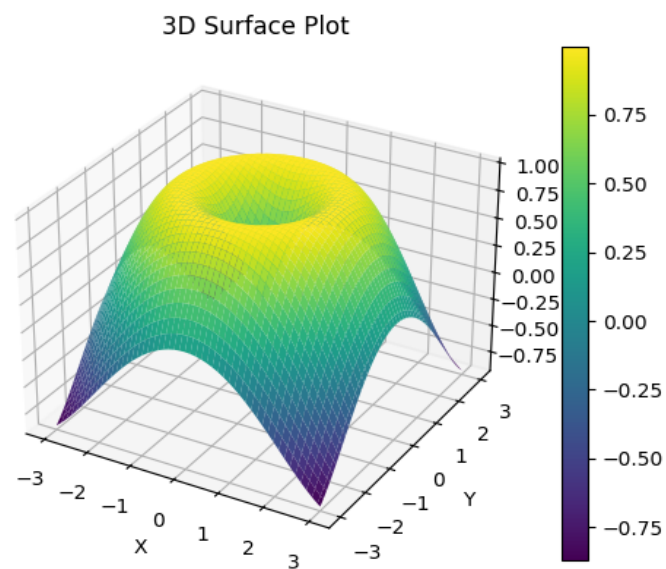
```

# Enter node name: D
# Enter neighbors of D separated by comma:
#
# Enter node name: E
# Enter neighbors of E separated by comma: G
#
# Enter node name: F
# Enter neighbors of F separated by comma:
#
# Enter node name: G
# Enter neighbors of G separated by comma:
#
# Enter heuristic for A: 6
# Enter heuristic for B: 4
# Enter heuristic for C: 5
# Enter heuristic for D: 3
# Enter heuristic for E: 2
# Enter heuristic for F: 4
# Enter heuristic for G: 0
#
# Enter start node: A
# Enter goal node: G

# Visiting: A
# Visiting: B
# Visiting: E
# Visiting: G

# Path found: A -> B -> E -> G

```



.\2\surfacePlot.py

```
import numpy as np
import matplotlib.pyplot as plt

x = y = np.linspace(-3, 3, 100)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

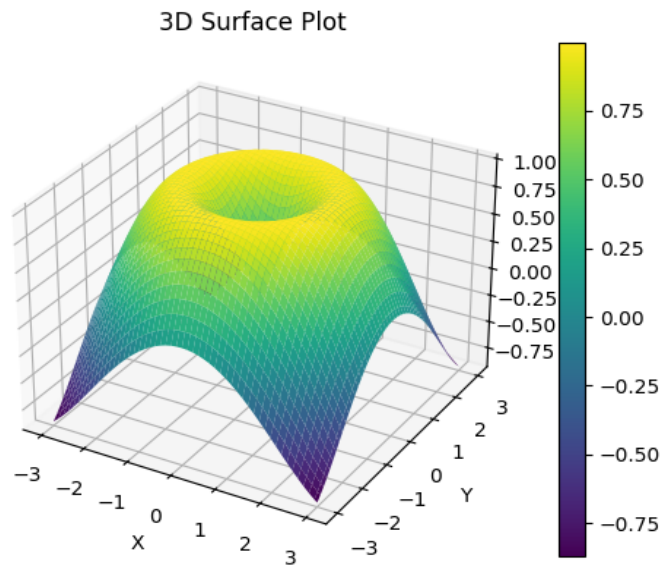
fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')

surf = ax.plot_surface(X, Y, Z, cmap='viridis')

fig.colorbar(surf)

ax.set_title("3D Surface Plot")
ax.set_xlabel("X")
ax.set_ylabel("Y")
ax.set_zlabel("Z")

plt.show()
```



.\3\b.py

```
from queue import PriorityQueue

def a_star(graph, heuristic, start, goal):
    pq = PriorityQueue()

    # (f, g, current_node, path)
    pq.put((heuristic[start], 0, start, [start]))

    visited = set()

    while not pq.empty():
        f, g, current, path = pq.get()

        if current in visited:
            continue

        visited.add(current)

        if current == goal:
            return path, g

        for neighbor, cost in graph[current]:
            if neighbor not in visited:
                new_g = g + cost
                new_f = new_g + heuristic[neighbor]
                pq.put((new_f, new_g, neighbor, path + [neighbor]))

    return None, None

# Input Graph
graph = {}

n = int(input("Enter number of nodes: "))

for i in range(n):
    node = input("Enter node name: ")
    graph[node] = []

    m = int(input(f"Enter number of neighbours of {node}: "))

    for j in range(m):
        neighbor = input("Enter neighbour name: ")
        cost = int(input("Enter cost: "))
        graph[node].append((neighbor, cost))

# Heuristic values
heuristic = {}

for node in graph:
    heuristic[node] = int(input(f"Enter heuristic value of {node}: "))

start = input("Enter start node: ")
goal = input("Enter goal node: ")

path, total_cost = a_star(graph, heuristic, start, goal)

if path:
    print("\nPath Found:")
    print(" -> ".join(path))
    print("Total Cost =", total_cost)
else:
    print("No path found")

"""Enter number of nodes: 6

Enter node name: A
Enter number of neighbours of A: 2
Enter neighbour name: B
Enter cost: 1
Enter neighbour name: C
Enter cost: 4

Enter node name: B
Enter number of neighbours of B: 2
```

```

Enter neighbour name: D
Enter cost: 2
Enter neighbour name: E
Enter cost: 5

Enter node name: C
Enter number of neighbours of C: 1
Enter neighbour name: F
Enter cost: 3

Enter node name: D
Enter number of neighbours of D: 0

Enter node name: E
Enter number of neighbours of E: 1
Enter neighbour name: F
Enter cost: 1

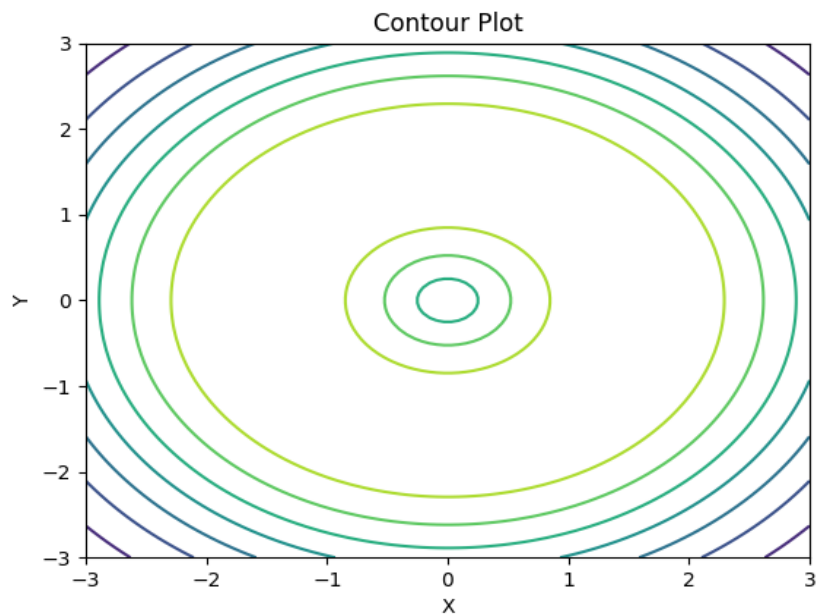
Enter node name: F
Enter number of neighbours of F: 0

Enter heuristic value of A: 7
Enter heuristic value of B: 6
Enter heuristic value of C: 2
Enter heuristic value of D: 1
Enter heuristic value of E: 1
Enter heuristic value of F: 0

Enter start node: A
Enter goal node: F

Path Found:
A -> B -> E -> F
Total Cost = 7""

```



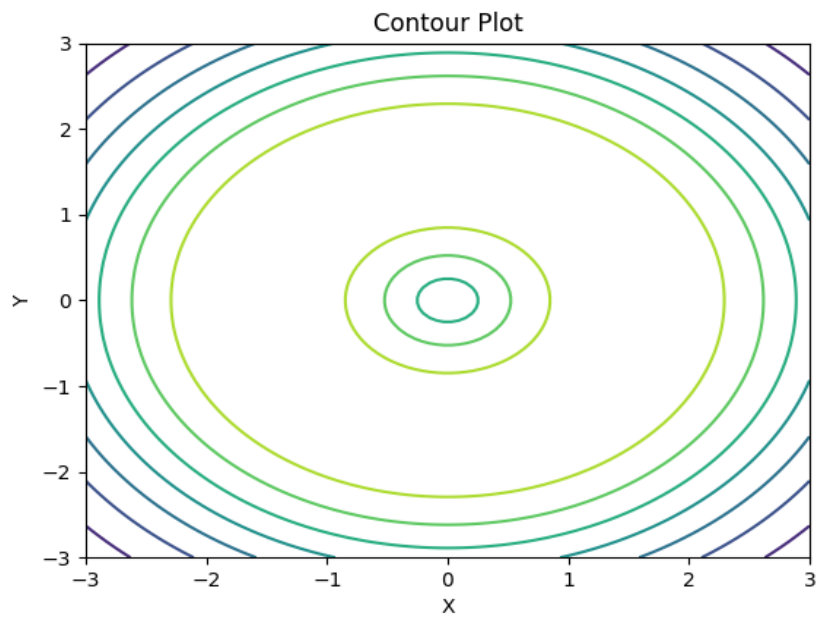
.\3\cont.py

```
import numpy as np
import matplotlib.pyplot as plt

x = y = np.linspace(-3, 3, 100)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

plt.contour(X, Y, Z)

plt.title("Contour Plot")
plt.xlabel("X")
plt.ylabel("Y")
plt.show()
```



.\4\heat.py

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

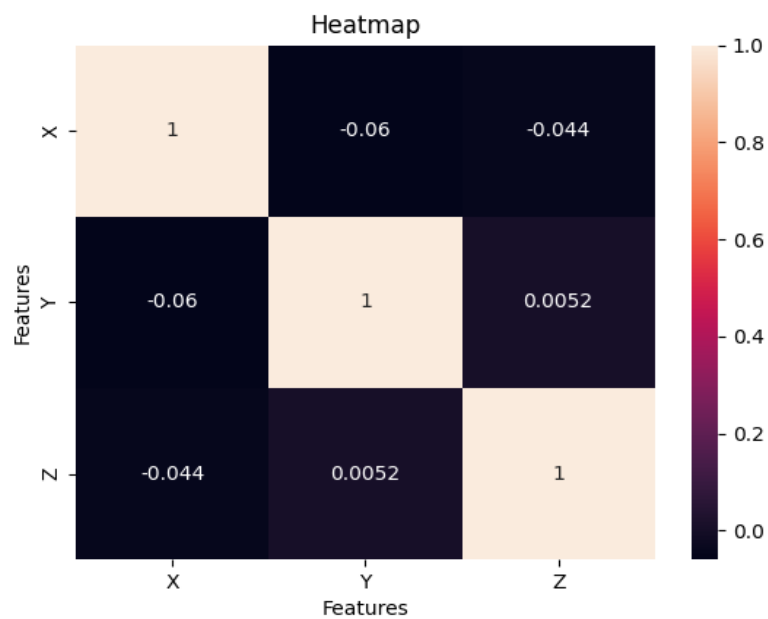
n = 100

data = pd.DataFrame()

for c in ['X', 'Y', 'Z']:
    data[c] = np.random.normal(size=n)

sns.heatmap(data.corr(), annot=True)

plt.title("Heatmap")
plt.xlabel("Features")
plt.ylabel("Features")
plt.show()
```



.\4\min.py

```
def minimax(depth, node, is_max, scores, max_depth):
    if depth == max_depth:
        return scores[node]

    left = minimax(depth + 1, node * 2, not is_max, scores, max_depth)
    right = minimax(depth + 1, node * 2 + 1, not is_max, scores, max_depth)

    return max(left, right) if is_max else min(left, right)

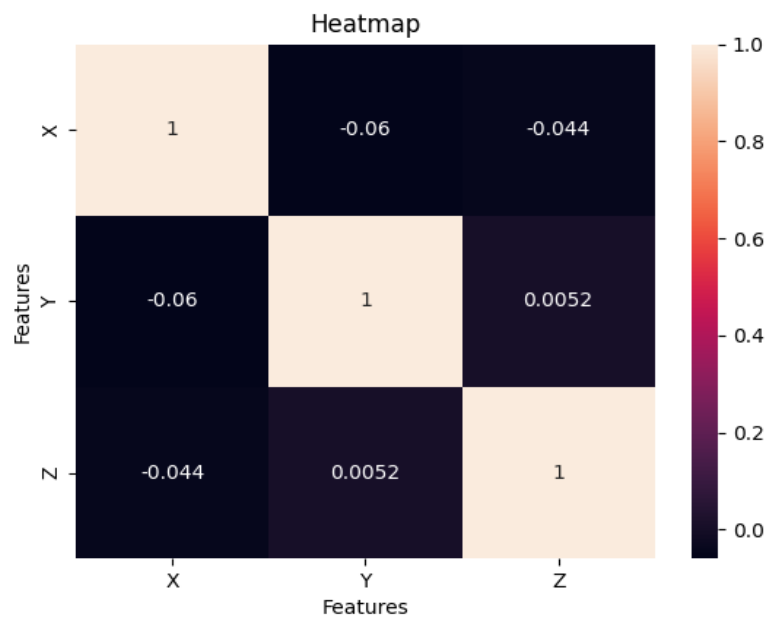
depth = int(input("Enter tree depth: "))

num_leaves = 2 ** depth
print(f"Enter {num_leaves} leaf node scores:")

scores = list(map(int, input().split()))

if len(scores) != num_leaves:
    print("Invalid number of scores")
else:
    result = minimax(0, 0, True, scores, depth)
    print("Optimal value using Minimax:", result)

"""Enter tree depth: 3
Enter 8 leaf node scores:
3 5 2 9 12 5 23 23
Optimal value using Minimax: 12
"""
```



.\5\alphabeta.py

```
def alphabeta(depth, node, is_max, scores, max_depth, alpha, beta):
    if depth == max_depth:
        return scores[node]

    value = float('-inf') if is_max else float('inf')

    for i in range(2):
        child = alphabeta(
            depth + 1,
            node * 2 + i,
            not is_max,
            scores,
            max_depth,
            alpha,
            beta
        )

        if is_max:
            value = max(value, child)
            alpha = max(alpha, value)
        else:
            value = min(value, child)
            beta = min(beta, value)

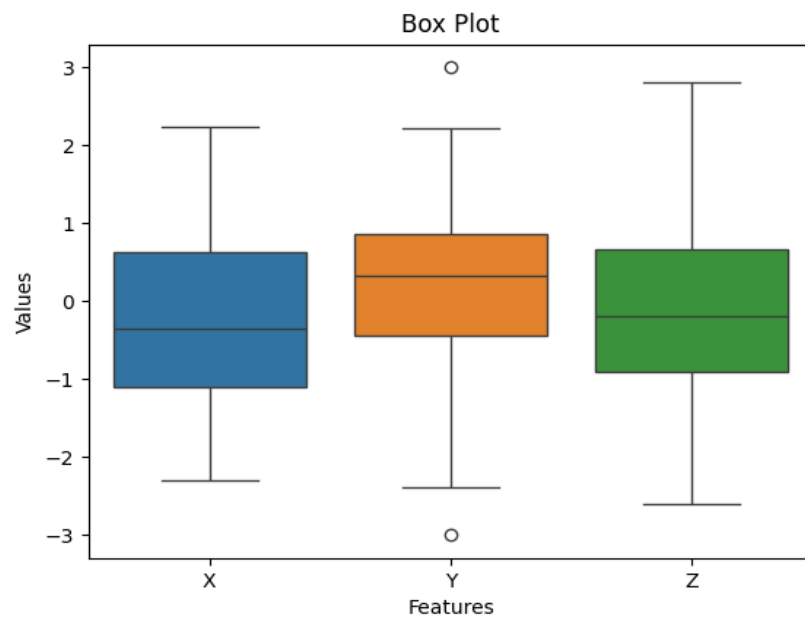
        if beta <= alpha:
            break

    return value

depth = int(input("Enter tree depth: "))
scores = list(map(int, input("Enter leaf scores: ").split()))

if len(scores) == 2 ** depth:
    print("Optimal value:",
          alphabeta(0, 0, True, scores, depth, float('-inf'), float('inf'))
    )
else:
    print("Invalid number of scores")

"""
Enter tree depth: 3
Enter leaf scores: 3 5 2 9 12 5 23 23
Optimal value: 12
"""
```



.\5\box.py

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

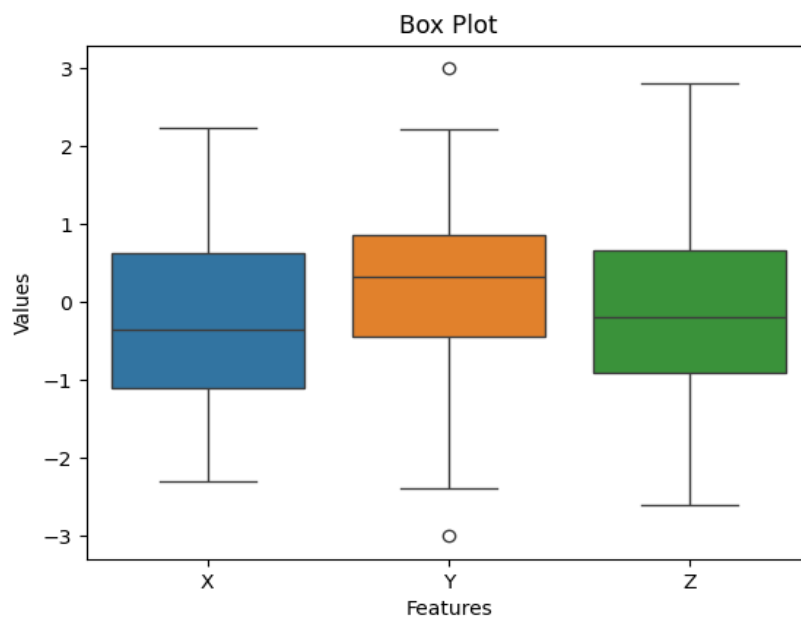
n = 100

data = pd.DataFrame()

for c in ['X', 'Y', 'Z']:
    data[c] = np.random.normal(size=n)

sns.boxplot(data=data)

plt.title("Box Plot")
plt.xlabel("Features")
plt.ylabel("Values")
plt.show()
```



.\6\b.py

```
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score, confusion_matrix

df = sns.load_dataset("titanic")

df = df[['pclass', 'sex', 'age', 'sibsp', 'parch', 'fare', 'survived']]

df['age'] = df['age'].fillna(df['age'].median())
df['sex'] = df['sex'].map({'male': 0, 'female': 1})

X = df.drop('survived', axis=1)
y = df['survived']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.1, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print(f"Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))

"""
Accuracy: 81.11%

Confusion Matrix:
[[44 10]
 [ 7 29]]
"""
```

.\7\1a.py

```
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

glass = fetch_openml("glass", as_frame=True)

X = glass.data
y = glass.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

for metric in ['euclidean', 'manhattan']:
    knn = KNeighborsClassifier(n_neighbors=3, metric=metric)

    knn.fit(X_train, y_train)

    y_pred = knn.predict(X_test)

    print(f"\nKNN using {metric.capitalize()} Distance")
    print(f"Accuracy: {accuracy_score(y_test, y_pred) * 100:.2f}%")
    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))

# KNN using Euclidean Distance
# Accuracy: 69.23%
# Confusion Matrix:
# [[14  3  0  0  0  1]
#  [ 8 18  1  0  0  0]
#  [ 0  1  4  0  0  0]
#  [ 0  2  0  7  1  0]
#  [ 0  0  0  0  1  0]
#  [ 3  0  0  0  0  1]]

# KNN using Manhattan Distance
# Accuracy: 72.31%
# Confusion Matrix:
# [[16  1  0  0  0  1]
#  [ 7 18  2  0  0  0]
#  [ 0  1  4  0  0  0]
#  [ 0  2  0  7  1  0]
#  [ 0  0  0  0  1  0]
#  [ 3  0  0  0  0  1]]
```

.\8\1a.py

```
import numpy as np
from sklearn.datasets import load_iris
import matplotlib.pyplot as plt

# Load Iris Dataset
iris = load_iris()
X = iris.data

k = 3

# Select first k points as initial centroids
centroids = X[:k]

for _ in range(100):

    # Calculate distances from each point to each centroid
    distances = np.sqrt(((X[:, np.newaxis] - centroids) ** 2).sum(axis=2))

    # Assign points to nearest centroid
    labels = np.argmin(distances, axis=1)

    # Compute new centroids
    new_centroids = []

    for i in range(k):
        cluster_points = X[labels == i]
        new_centroids.append(cluster_points.mean(axis=0))

    new_centroids = np.array(new_centroids)

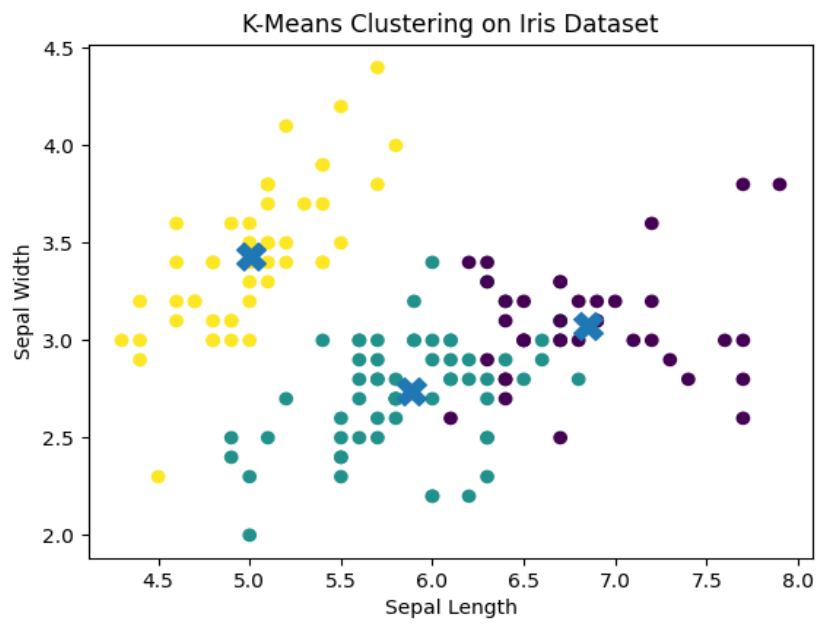
    # Stop if centroids do not change
    if np.allclose(centroids, new_centroids):
        break

    centroids = new_centroids

print("\nFinal Centroids:")
for i in range(k):
    print(f"Cluster {i+1} Centroid: {centroids[i]}")

plt.scatter(X[:, 0], X[:, 1], c=labels)
plt.scatter(centroids[:, 0], centroids[:, 1], marker='x', s=200)
plt.xlabel("Sepal Length")
plt.ylabel("Sepal Width")
plt.title("K-Means Clustering on Iris Dataset")
plt.show()

# Final Centroids:
# Cluster 1 Centroid: [6.85384615 3.07692308 5.71538462 2.05384615]
# Cluster 2 Centroid: [5.88360656 2.74098361 4.38852459 1.43442623]
# Cluster 3 Centroid: [5.006 3.428 1.462 0.246]
```



.\9\9q.py

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.datasets import load_iris

iris = load_iris()
data = iris.data[:6]

def proximity_matrix(data):
    n = data.shape[0]
    proximity_matrix = np.zeros((n, n))
    for i in range(n):
        for j in range(i+1, n):
            proximity_matrix[i, j] = np.linalg.norm(data[i] - data[j])
            proximity_matrix[j, i] = proximity_matrix[i, j]
    return proximity_matrix

def plot_dendrogram(data, method):
    linkage_matrix = linkage(data, method=method)
    dendrogram(linkage_matrix)
    plt.title(f'Dendrogram - {method} linkage')
    plt.xlabel('Data Points')
    plt.ylabel('Distance')
    plt.show()

print("Proximity matrix:")
pm = proximity_matrix(data)
print(proximity_matrix(data))

plot_dendrogram(data, 'single')
plot_dendrogram(data, 'complete')

# Proximity matrix:
# [[0.         0.53851648 0.50990195 0.64807407 0.14142136 0.6164414 ]
#  [0.53851648 0.         0.3         0.33166248 0.60827625 1.09087121]
#  [0.50990195 0.3         0.         0.24494897 0.50990195 1.08627805]
#  [0.64807407 0.33166248 0.24494897 0.         0.64807407 1.16619038]
#  [0.14142136 0.60827625 0.50990195 0.64807407 0.         0.6164414 ]
#  [0.6164414  1.09087121 1.08627805 1.16619038 0.6164414  0.         ]]
```

